

Chapter 09 단순 인증 프로토콜

9.1 개요

➤ 일반적으로 **프로토콜**은 의사소통 규칙을 뜻함

▪ 예: 수업 중에 질문을 해야 할 때 프로토콜

- ① 손을 든다.
- ② 선생님이 이름을 부른다
- ③ 질문을 한다.
- ④ 선생님이 대답한다.

➤ 보안 프로토콜은 보안 응용 프로그램에서 따르는 통신 규칙

- SSH, SSL/TLS, IPsec, WEP, WPA 등
- 이 장에서는 단순화된 규모가 작은 인증 프로토콜에 대해 다룸

➤ 보안 프로토콜의 기본적인 필수 요소들을 알아봄으로써 보안 프로토콜의 설계와 관련된 근본적인 보안 문제들을 더 잘 이해할 수 있을 것

9.2 단순 보안 프로토콜

➤ 보안 출입 프로토콜

- 보안 시설에 출입하는 데 사용되는 프로토콜

- ① 출입증을 판독기에 넣는다.
- ② PIN을 입력한다.
- ③ PIN이 정확한가?
 - 네 → 건물에 들어간다
 - 아니오 → 경비원한테 잡힌다

➤ IFF(Identify Friend or Foe)프로토콜

- 아군과 적군을 구별하는 프로토콜

- 본의 아니게 아군을 공격하는 행위인 오인 사격을 방지하기 위해 설계됨

9.2 단순 보안 프로토콜

■ IFF 프로토콜 예시

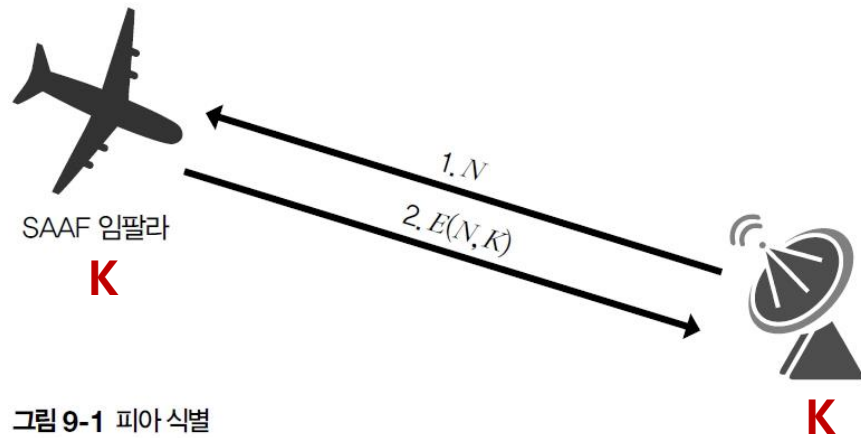
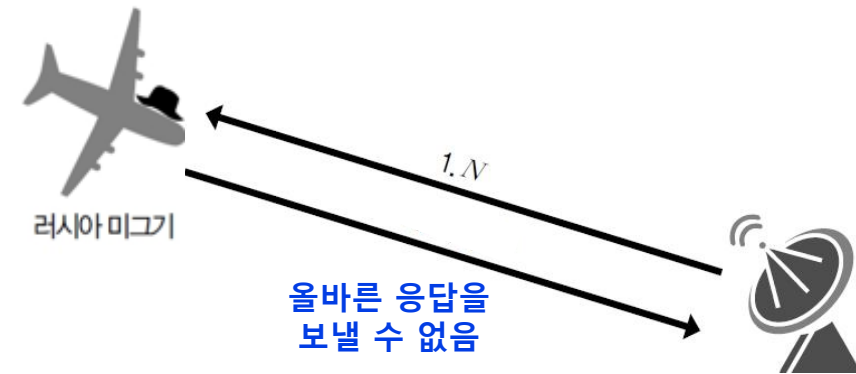


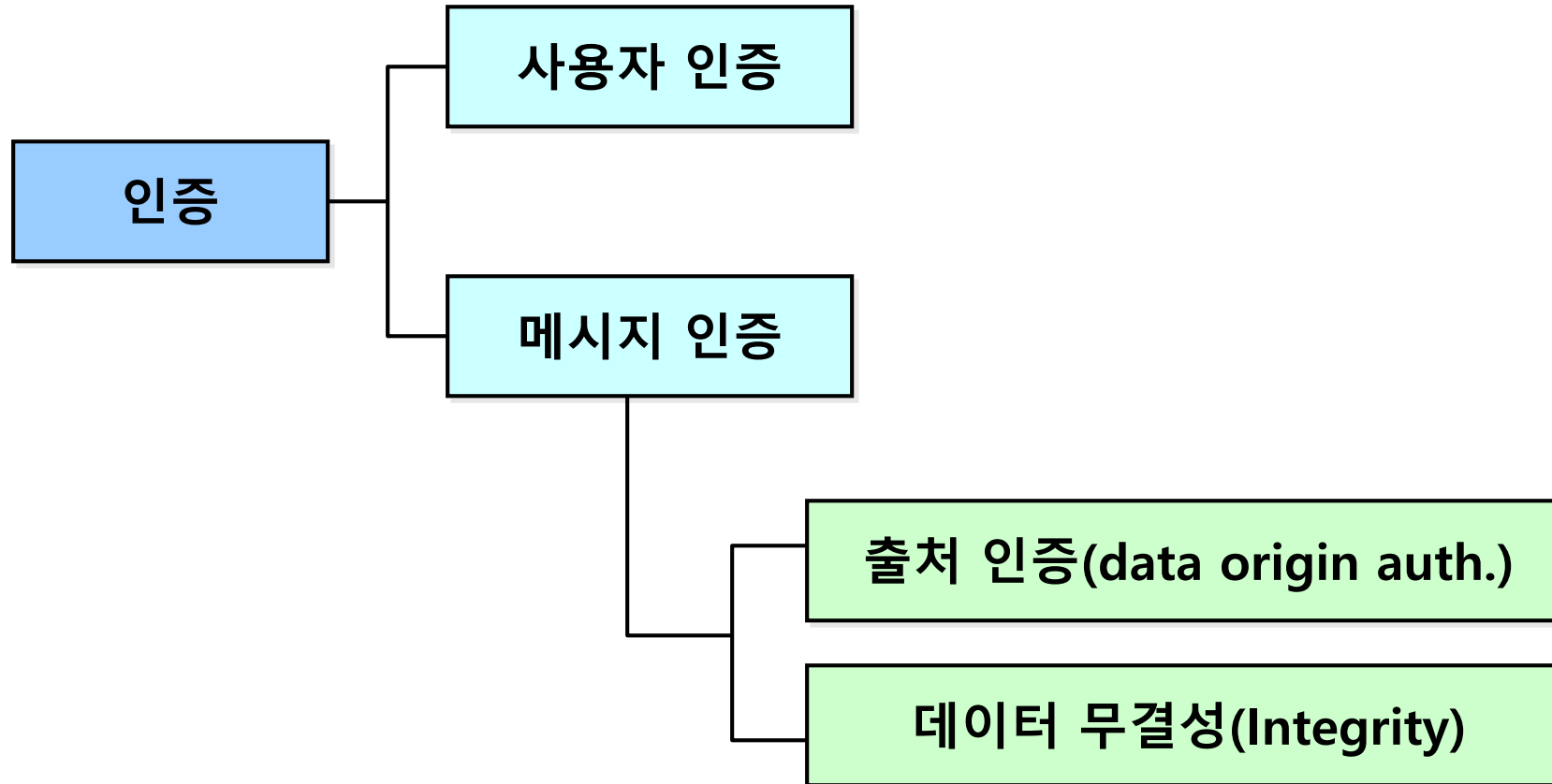
그림 9-1 피아 식별

K를 알지 못하는 적군



- E() : 대칭키 암호 알고리즘
- N : 난수
- K : 아군들이 공유하는 비밀키

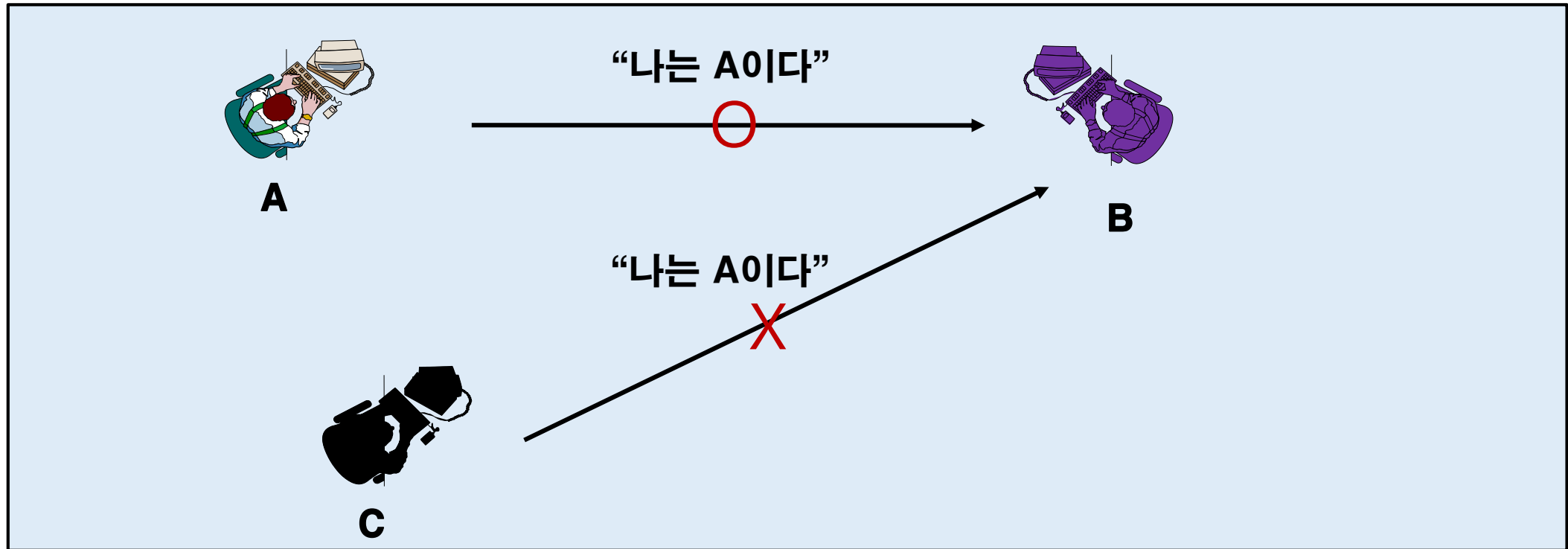
9.3 인증 프로토콜



9.3 인증 프로토콜

➤ 사용자 인증

- 사용자 A가 사용자 B에게 자신이 사용자 A임을 증명
- A를 제외한 다른 사람은 자신이 A인척 할 수 없어야 함



9.3 인증 프로토콜

- 엘리스와 밥이 네트워크에서 통신하고 있고 엘리스는 밥에게 자신이 엘리스라는 것을 증명해야 한다고 가정
 - 인증과 더불어 대부분의 경우 세션키(엘리스와 밥 사이의 공유 비밀키) 구축이 이루어짐
 - 인증에 성공한다면 세션키는 현재 세션의 기밀성과 무결성을 보호하는 데 사용
- 네트워크에서 사용자를 인증을 하기 위해 사용하는 간단한 프로토콜 예



9.3 인증 프로토콜

➤ [그림 9-3] 간단한 인증 프로토콜의 취약점

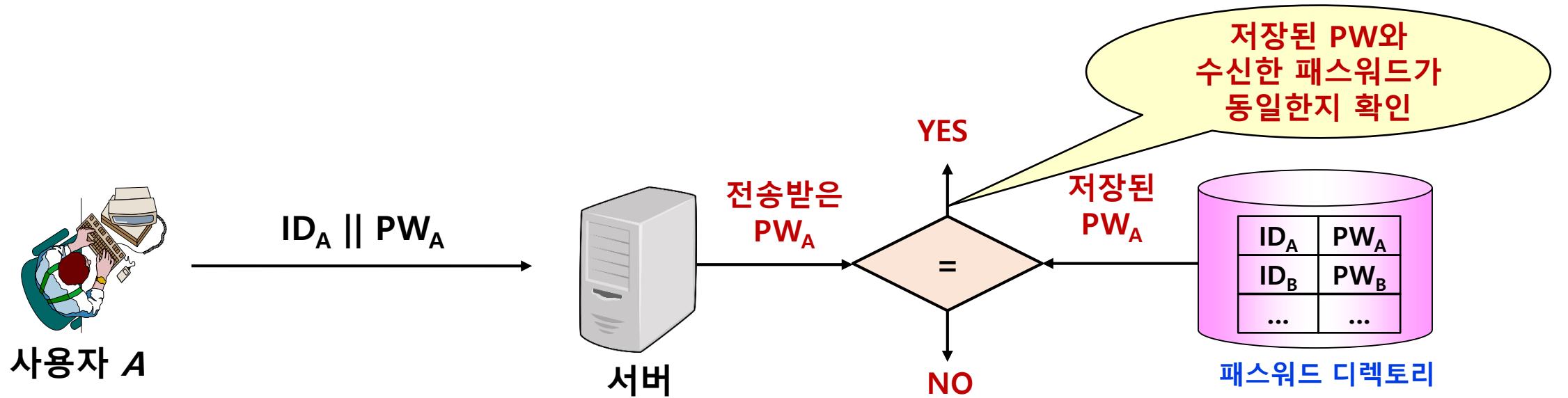
- 앨리스가 네트워크를 통해 전송하는 메시지를 트루디가 도청할 수 있다면 이 정보를 이용하여 트루디가 앨리스로 위장 가능(단순 재연 공격(replay attack))



그림 9-4 단순 재연 공격

- 앨리스의 **패스워드가 평문으로 전송**
 - 앨리스가 여러 사이트에서 특정 패스워드를 반복 사용한다면 트루디 역시 해당 사이트들에서 앨리스 행세를 할 수 있음

9.3 인증 프로토콜



❖ 문제점 ?

- 패스워드 노출 ➡ 도청에 의한 재전송 공격(replay attack) 가능
- 서버의 패스워드 디렉토리 보안 문제
- 패스워드의 길이가 짧은 경우, 사전 공격(dictionary attack)에 취약

사전에 있는 단어를 순차적으로 입력하여 PW를 찾는 공격 기법

9.3 인증 프로토콜

➤ 해시 함수를 이용하는 인증 프로토콜 예



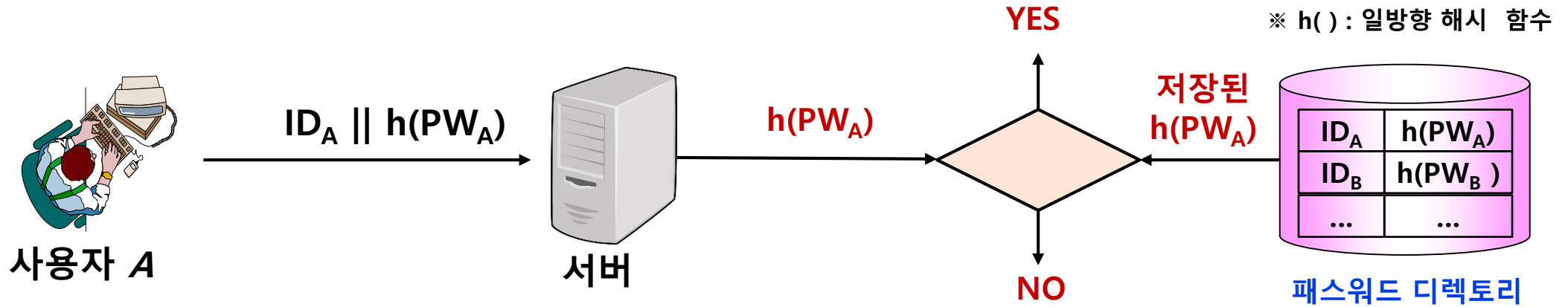
그림 9-5 해시가 있는 단순 인증

- 앨리스의 비밀번호 평문 상태로 전송되는 취약점은 해결
- 트루디가 $h(\text{앨리스의 비밀번호})$ 를 도청하여 재연 공격을 시도하는 경우, 앨리스로 위장 가능(재연 공격 가능) → 앨리스가 인증을 위해 전송하는 정보가 매번 동일하기 때문에 이러한 공격이 가능해짐

※ 밥(서버)은 인증 시마다 다른 질문을 전송하고, 앨리스 만이 정확한 답변을 제공할 수 있는 인증 방식
필요 → 질문-응답(Challenge-Response) 방식

9.3 인증 프로토콜

➤ 해시 함수를 이용하는 인증 프로토콜



❖ 문제점 ?

- 도청에 의한 재전송 공격 가능
- 패스워드의 길이가 짧은 경우, 사전 공격(dictionary attack)에 취약

9.3 인증 프로토콜

➤ 간단한 인증 프로토콜([그림 9-3])의 특징

- 사용자의 ID와 password를 이용한 사용자 인증
- 동일한 패스워드를 이용한 인증 방식의 재연 공격에 취약
- 사용자의 패스워드 노출 및 도용을 방지할 수 있는 인증 기술 필요

➤ 질문-응답(Challenge-Response) 방식에 의한 인증

- 서버는 사용자가 접속 시 마다 다른 질문 전송
- 정당한 사용자만이 질문에 대한 정확한 응답 제공 가능
- 질문은 재 사용되지 않음
- 도청이나 재전송 공격에 대해 안전
- 종류
 - 대칭키 암호 방식을 이용한 인증
 - 공개키 암호 방식을 이용한 인증

9.3 인증 프로토콜

➤ 질문-응답(Challenge-Response) 인증 프로토콜의 개념

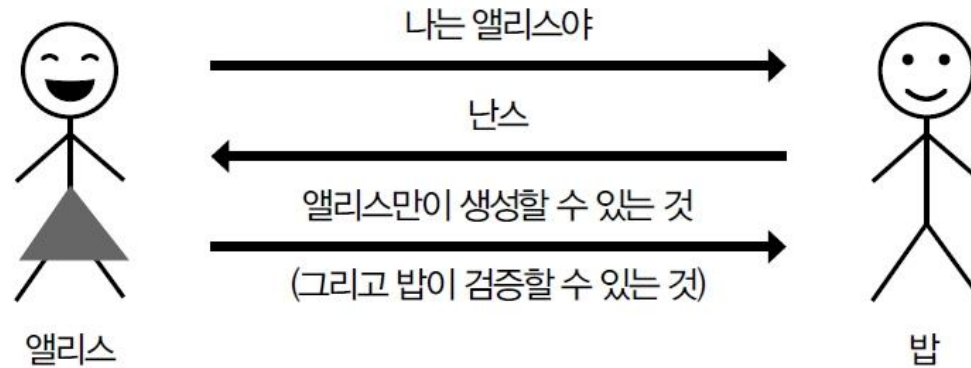


그림 9-6 일반적인 인증

난스(nonce)는 인증 시 마다
랜덤하게 생성되고 한번만 사용하는 값

※ Nonce : Number Only Used Once

9.3 인증 프로토콜

(1) 해시 함수를 이용하는 질문-응답 인증 프로토콜

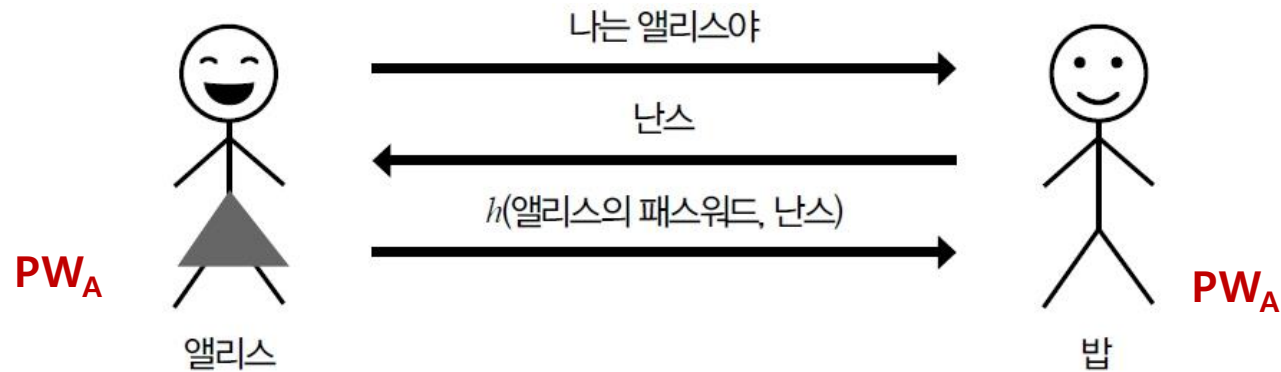


그림 9-7 질문-응답

밥은 저장된 앨리스의 패스워드와
난스(nonce)의 해시 값을 계산하고,
앨리스가 전송한 정보와 일치하는 지 확인

- ❖ Nonce는 한번 사용한 후 재사용되지 않으므로, 트루디가 앨리스의 응답 정보를 획득하더라도 다시 사용할 수 없음
 - ➡ 재전송 공격에 대해 안전

9.3 인증 프로토콜

(2) 대칭키 암호 방식을 이용한 질문-응답 인증 프로토콜

- 엘리스와 밥이 대칭키 K_{AB} 를 공유하고 있으며 엘리스와 밥 외에는 K_{AB} 에 접근할 수 없다고 가정
- 엘리스는 키를 트루디에게 노출시키지 않고 자신이 그 키를 알고 있다는 것을 증명함으로써 자신을 밥에게 인증할 것
- 프로토콜은 재연 공격으로부터 안전해야 함



그림 9-8 대칭키 인증 프로토콜

- $E()$: 대칭키 암호 알고리즘
- R : 난수
- K_{AB} : 엘리스와 밥의 공유 비밀키

- ❖ 질문 R 은 한번 사용한 후 재사용되지 않으므로, 트루디가 엘리스의 응답 정보를 획득하더라도 다시 사용할 수 없음
 - ➡ 재연 공격에 대해 안전

9.3 인증 프로토콜



- K_A : 사용자 A와 서버 사이에 사전에 공유한 비밀정보
- $E()/D()$: 대칭키 암호화/복호화 알고리즘

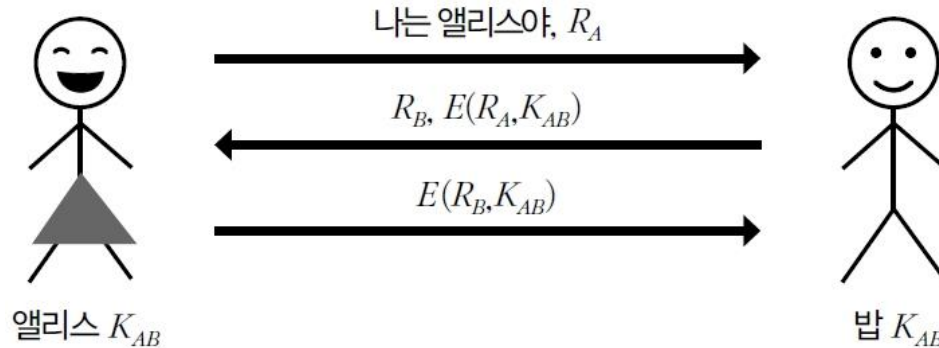
❖ 질문 값 r 은 한번 사용한 후 재사용되지 않으므로, 응답 R 을 도청하더라도 사용할 수 없음
➡ 도청에 의한 재전송 공격에 대해 안전

9.3 인증 프로토콜

- 대칭키 암호 방식을 이용한 상호 인증(Mutual authentication)
 - 앨리스와 밥이 서로를 인증하기 위해 한번은 밥이 앨리스를 인증하고 또 한번은 앨리스가 밥을 인증하도록 하는 과정을 두 번 반복

(1) 앨리스는 밥을 인증하기 위해 질문 R_A 전송

(3) 앨리스는 밥을 인증한 후, 응답 전송



(2) 밥은 응답과 함께 앨리스를 인증하기 위해 질문 R_A 전송

그림 9-10 보안상호 인증?

❖ 이 방식은 안전한 상호인증을 제공할까?

9.3 인증 프로토콜

- [그림 9-10] 상호 인증 방식의 취약점(반사 공격(reflection attack))
 1. 트루디는 밥에게 앨리스로 위장하여 질문 R_A 전송
 2. 밥은 응답 $E(R_A, K_{AB})$ 와 질문 R_B 전송
 3. 다른 연결을 통해 트루디는 앨리스로 위장하여 질문 R_B 전송
 4. 밥은 응답 $E(R_B, K_{AB})$ 와 새로운 질문 $\tilde{R}_B$ 전송
 5. 트루디는 밥의 응답 $E(R_B, K_{AB})$ 를 첫번째 인증 과정의 응답으로 전송 → 인증 성공

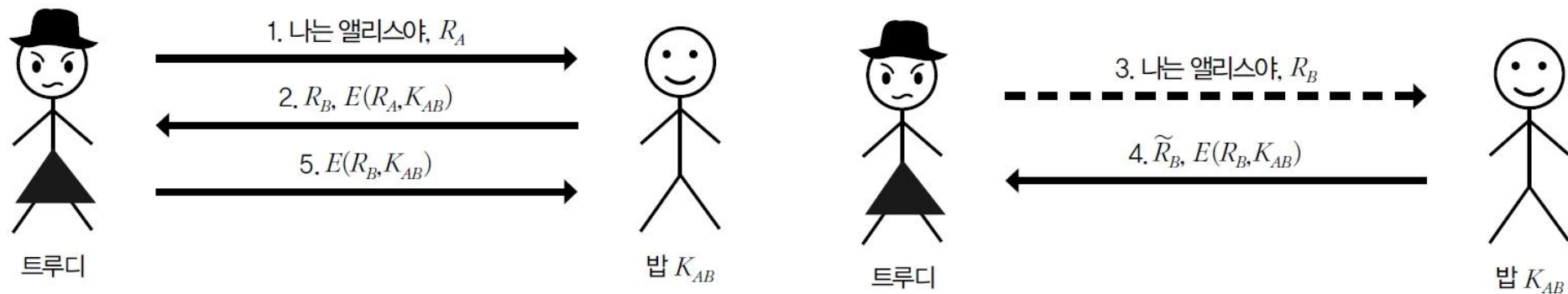


그림 9-11 트루디의 공격

밥이 보낸 질문을 앨리스가 보낸 것처럼 재전송하여 응답 값을 알아내는 공격

9.3 인증 프로토콜

■ 안전한 상호 인증 프로토콜

- 사용자의 **신분 정보(ID)**를 **응답을 생성하는 과정에 포함** 시킴
- 이 변화는 세 번째 메시지를 위해 트루디가 밥에게서 받은 응답을 사용할 수 없으므로 트루디의 이전 공격을 예방하는 데 효과적

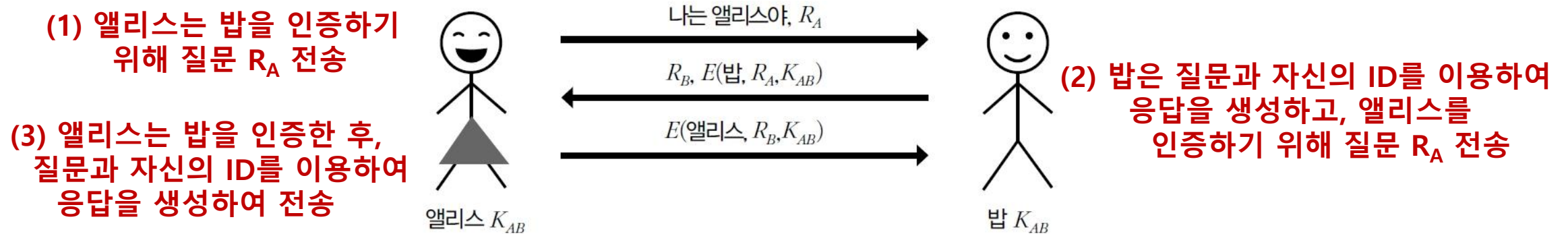
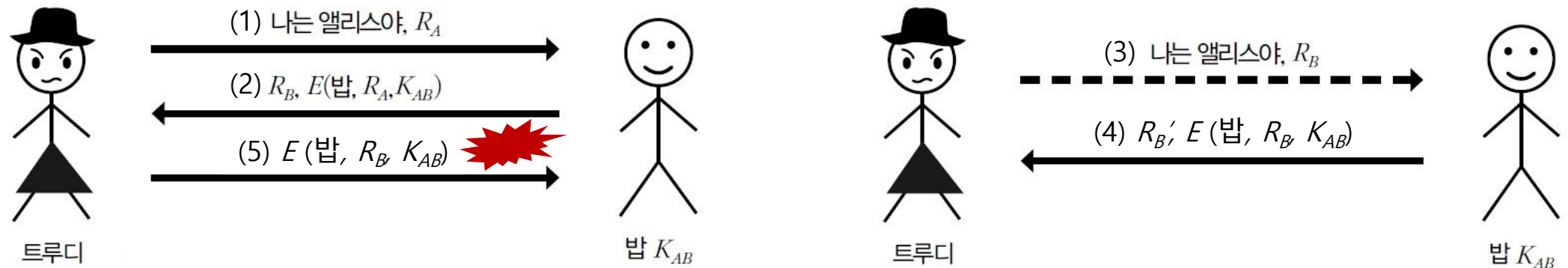


그림 9-12 강력한 상호 인증 프로토콜

9.3 인증 프로토콜

➤ 안전한 상호 인증 방식에 대한 반사 공격

1. 트루디는 밥에게 앨리스로 위장하여 질문 R_A 전송
2. 밥은 응답 $E(\text{밥}, R_A, K_{AB})$ 와 질문 R_B 전송
3. 다른 연결을 통해 트루디는 앨리스로 위장하여 질문 R_B 전송
4. 밥은 응답 $E(\text{밥}, R_B, K_{AB})$ 와 새로운 질문 R_B' 전송
5. 트루디는 밥의 응답 $E(\text{밥}, R_B, K_{AB})$ 를 첫번째 인증 과정의 응답으로 전송 → 앨리스의 ID가 아닌 밥의 ID가 포함되어 있는 응답 값이므로 인증 실패



9.3 인증 프로토콜

(3) 공개키 암호 방식을 이용한 질문-응답 프로토콜

- ① 앨리스가 밥에게 인증을 받기 위해 ID 전송
- ② 밥은 난수 R 을 생성하여 앨리스의 공개키로 암호화 한 질문 전송
- ③ 앨리스는 자신의 개인키를 이용하여 질문을 복호하여 응답 R 전송

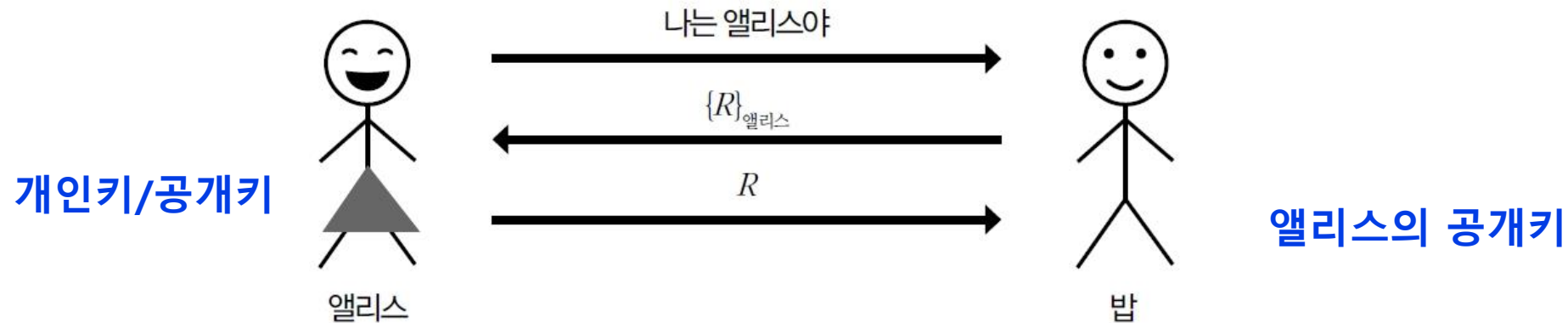


그림 9-13 공개키 암호화를 사용한 인증

- R : 난수
- $\{R\}_{\text{앨리스}}$: 앨리스의 공개키로 R 을 암호화

9.3 인증 프로토콜

(4) 디지털 서명을 이용한 질문-응답 인증 방식

- ① 앨리스가 밥에게 인증을 받기 위해 ID 전송
- ② 밥은 난수 R 을 생성 질문 전송
- ③ 앨리스는 자신의 개인키를 이용하여 질문 R 에 대한 디지털 서명 값을 응답으로 전송
- ④ 밥은 앨리스의 공개키로 서명을 검증하여 인증 수행

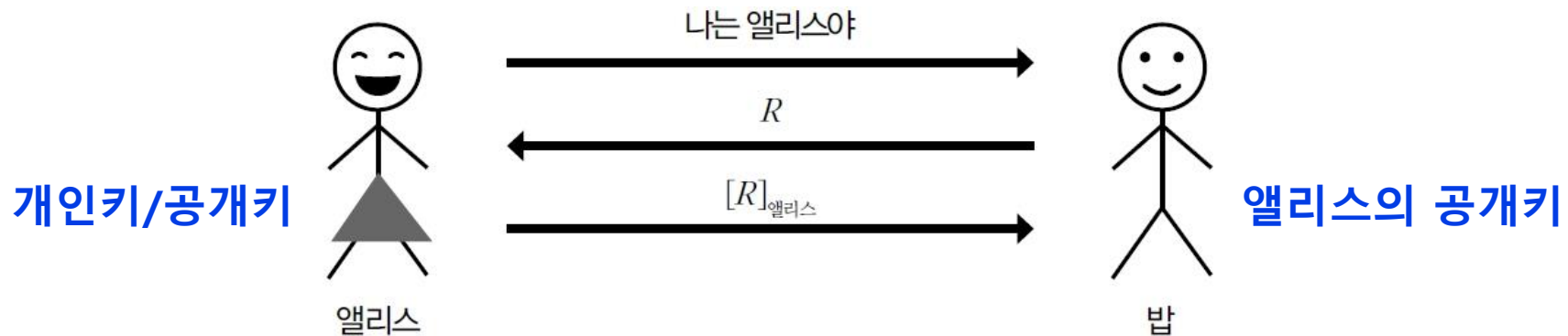


그림 9-14 디지털 서명을 통한 인증

- R : 난수
- $[R]_{\text{앨리스}}$: 앨리스의 개인키로 R 에 대한 서명 생성

9.3 인증 프로토콜

➤ 세션키(Session key)

- 인증을 위해 사용한 공유 비밀키(대칭키)가 있더라도, 각 세션 내에서 데이터를 암호화하기 위해서는 다른 키 이용(세션키)
- 어떤 특정 키로 암호화되는 데이터의 양을 제한하는 것이며 **하나의 세션키가 손상되었을 때 입을 수 있는 피해를 줄이고자** 하는 것
- 세션키는 기밀성이나 무결성을 제공하기 위해 사용

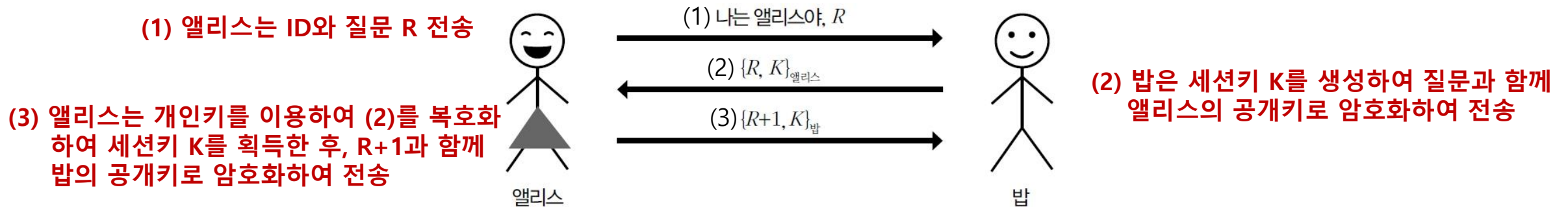


그림 9-15 인증과 세션키

[인증과 세션키 설정을 제공하는 프로토콜]

9.3 인증 프로토콜

- 디지털 서명 방식을 이용하는 인증과 세션키 설정을 제공하는 프로토콜

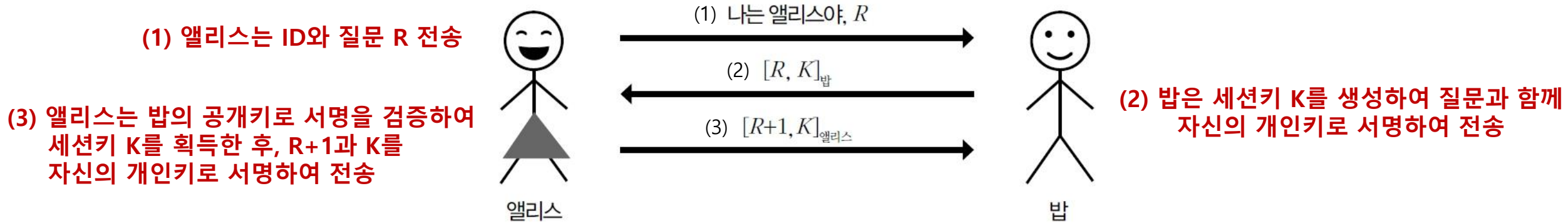


그림 9-16 서명에 기반을 둔 인증과 세션키

- [그림 9-16]의 프로토콜에는 한 가지 심각한 결함이 존재
 - 디지털 서명을 이용하여 **서명된 키 값은 기밀성을 제공하지 못함**
 - 공개된 세션키는 절대적으로 안전하지 않음

9.3 인증 프로토콜

■ [그림 9-16] 방식의 취약점을 해결하기 위해 암호화 후 서명하는 방식([그림 9-18])

- (1) 앨리스는 ID와 질문 R 전송
- (2) 밥은 세션키 K 를 생성하여 앨리스의 질문 값과 함께 앨리스의 공개키로 암호화 한 후, 자신의 개인키로 서명하여 전송
- (3) 앨리스는 (2)의 서명을 검증한 후, 자신의 개인키로 복호화하여 세션키 K 획득. $R+1$ 과 세션키 K 를 밥의 공개키로 암호화 한 후, 자신의 개인키로 서명하여 밥에게 전송

(1) 앨리스는 ID와 질문 R 전송

(3) 앨리스는 밥의 공개키로 서명을 검증한 후, 자신의 개인키로 복호화하여 세션키 K 획득. $R+1$ 과 K 를 밥의 공개키로 암호화 한 후, 자신의 개인키로 서명하여 전송

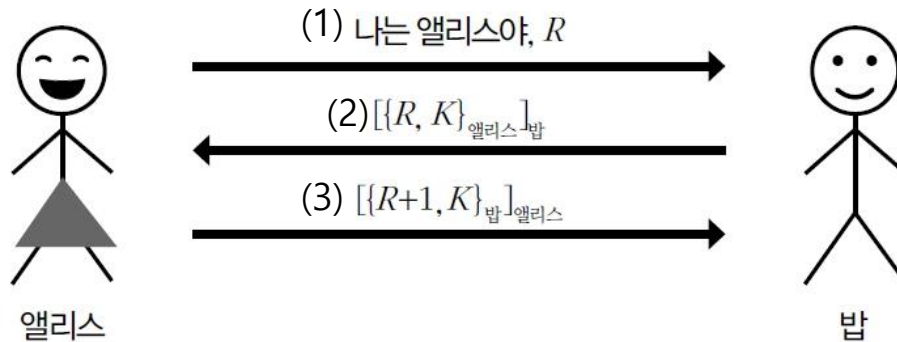


그림 9-18 암호화 및 서명 상호 인증

(2) 밥은 세션키 K 를 생성하여 질문과 함께 앨리스의 공개키로 암호화 한 후, 자신의 개인키로 서명하여 전송

(4) 밥은 앨리스의 서명을 검증한 후, 자신의 개인키로 복호화하여 $R+1$ 과 세션키 K 확인

9.3 인증 프로토콜

➤ 완전한 전방향 안전성(PFS: Perfect Forward Secrecy)

- 세션키를 생성하기 위해 사용한 longterm key가 노출되더라도 세션키(session key)의 안전성을 보장하는 특징
 - **longterm key** : 장기간 사용하는 비밀키 (예) 클라이언트나 서버의 개인키)
 - **session key** : 하나의 통신 연결(세션)을 암호화하기 위해 사용하는 비밀키(예) 대칭키 암호 방식에 사용하는 공유 비밀키



그림 9-19 PFS 구현을 위한 순진한 시도

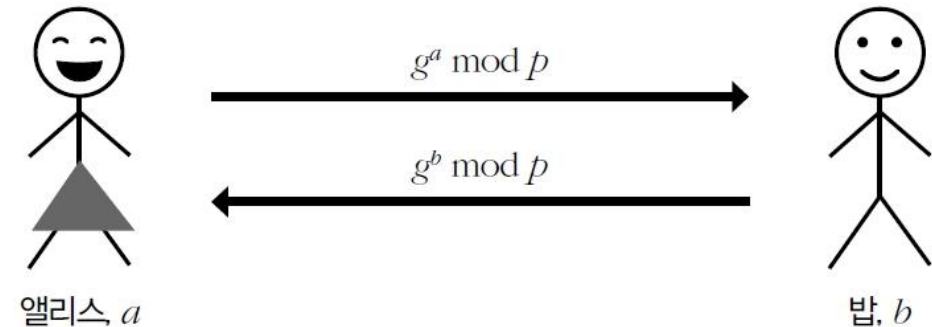


그림 9-20 디파-헬먼

9.3 인증 프로토콜

- 만약 PFS를 구현하기 위해 디피-헬먼을 사용한다면 중간자 공격을 극복할 수 있어야 함
 - 중간자 공격을 예방하기 위해 앨리스와 밥은 공유 비밀키 K_{AB} 를 이용하여 전송하는 디피-헬먼 키 교환을 위한 공개키를 암호화하여 전송
 - 그리고 PFS를 제공하기 위해 앨리스와 밥은 해당 세션의 메시지 암호화에 사용할 공유 세션키 $KS \equiv g^{ab} \bmod p$ 생성

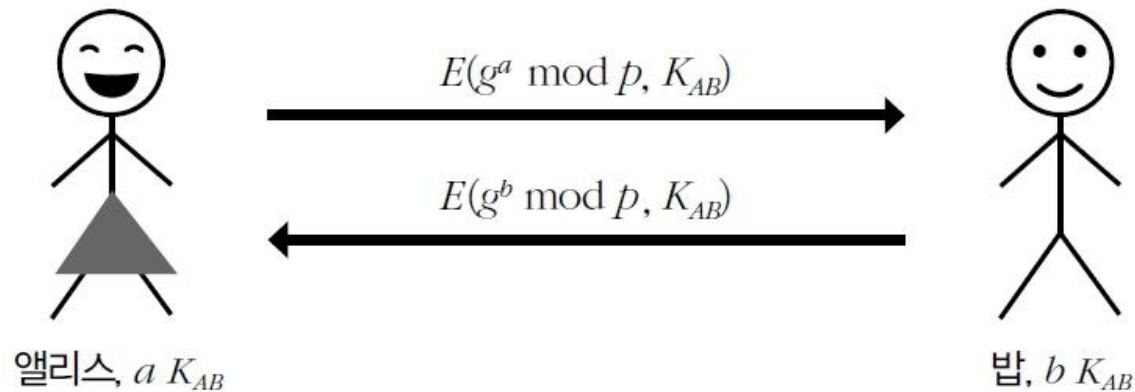


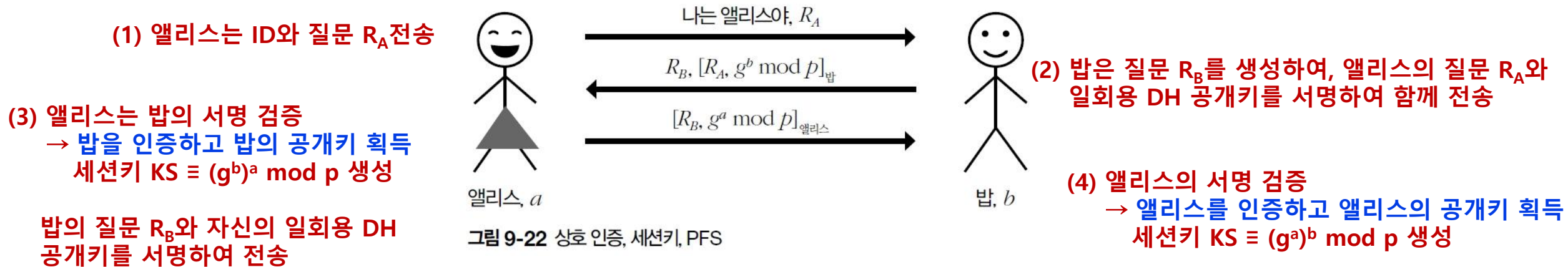
그림 9-21 PFS 구현을 위한 일회성 디피-헬먼

- **longterm key** : 앨리스와 밥 사이의 공유 비밀키 K_{AB}
- **session key** : 일회성 디피-헬먼 공유키

9.3 인증 프로토콜

➤ 상호 인증, 세션키 교환 및 PFS를 제공하는 프로토콜

▪ [그림 9-18]의 암호화 후 서명 프로토콜을 수정한 버전



- 엘리스와 밥은 상호 인증 가능
- 전송하는 DH 공개키의 무결성을 확인할 수 있으므로 중간자 공격에 대해 안전
- 엘리스와 밥의 서명용 개인키가 노출되더라도 세션키는 안전 → PFS 제공

9.6 프로토콜 분석을 위한 팁

- 가능한 공격의 종류가 많지만 대부분은 다음 네 가지 범주 중 하나에 속함
 - 재연 공격(Replay attack)
 - 반사 공격(Reflection attack)
 - 대체 공격(Replacement attack)
 - 중간자 공격(Man-in-the-middle attack)
- 이러한 공격들에 대해 취약하지 않도록 보안 프로토콜을 설계해야 함

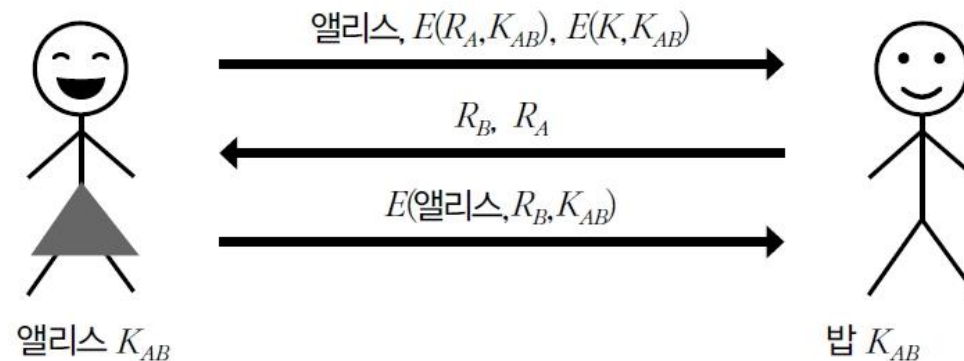


그림 9-33 대체 공격에 취약한 프로토콜